

Introduction

! This is a draft version. Latest published version available at [UAPI](#)

This document is an API specification template for all Lithuanian government institutions.

Institutions implementing API should reuse most of the specification as is and only add customizable parts expressed as path parameters or schema fields specified as a pattern.

This specification should be used together with [DSA](#) specification.

UAPI provides a template for data exchange protocol and DSA provides schemas.

Final API specification should be generated by combining UAPI and DSA together and this (in future) can be done in Data Portal ([data.gov.lt](#)).

Specifications

- [UAPI draft](#) - this document, describes data exchange protocol and format.
- [DSA](#) - Data Structure Mapping table format specification, describes a table format, for mapping data models between Physical, Logical and Conceptual data models.

Logical data model from DSA tables are used as API schemas. UAPI only describes protocol, DSA tables provides data schemas for UAPI API.

DSA and UAPI are used together.

- [DCAT-AP-LT](#) - a semantic data model specification, based on BregDCAT-AP.

Contact Details

This document is maintained by Lithuanian [State Digital Solutions Agency](#) (VSSA).

If you found an error in the document, you can report it by registering a [GitHub issue](#).

Legal basis

This specification is provided in accordance to

- [Lietuvos respublikos valstybės informacinių išteklių valdymo įstatymas \(XI-1807\)](#),

- [Dėl duomenų teikimo formatų ir standartų rekomendacijų patvirtinimo \(T-36\)](#).

Architecture

Diagram above provides a high level overview listing main components involved in data exchange.

This document mainly describes data exchange protocol which is implemented by **Data Agent**.

Components

Data exchange and interoperability architecture consists of following main components:

- **Data** source and **Data Service** or **Resource** when referred to one or the other - these components represents all data sources, including various databases, web services, CSV, XLSX or other data files maintained by all the government institutions.

Data might be read directly from a primary data source or through a data service.

- **Data agent** - is a software component similar to **Data Service** but publishes data in a standardized UAPI protocol.

VSSA provides an [Universal Data Agent](#), which uses DSA table to convert between data provided by a Resource into UAPI protocol.

Organizations are free to implement their own Data Agent solution according to UAPI specification.

- **Data Catalog** - is used cataloging datasets from all public institutions. Data Catalog is built on top of [DCAT-AP](#) data model and provides user interface for managing DSA tables.

Data Catalog is also used to manage smart data contracts.

Data Catalog receives DSA tables from Data Agent, provides modified versions of DSA tables back to Data Agent, exports API specifications to API Gateway in OpenAPI format and exports available data scopes and information about smart contracts into Auth service.

Data Catalog does not have access to data, it is used only to manage metadata about data.

- **API Gateway** - is used as a reverse proxy and passes Client requests to one of many Data agent. Data Agents can't be accessed directly, they are available only through API Gateway.

API Gateway provides load balancing, rate limiting, request validation against OpenAPI schemas, access token validation with Auth service and request logging and metrics.

- **Auth service** - provides client and user authentication and authorization services, supporting both OAuth [Client Credentials](#) and [Authorization Code](#) flows.

Diagram is divided in two data and metadata flows.

Metadata flow

First **metadata flow** is needed to produce a final API specification in OpenAPI format. This involves following steps:

1. **Data Agent** extracts data structure from Data source or Data Service in [DSA](#) table format and passes it to **Data Catalog**.

At the same time, **Data Agent** receives updated DSA version from **Data Catalog**.

This process is automated and DSA metadata exchange is done periodically (for example once every day).

2. Data steward from each organization manually edits DSA tables using **Data Catalog** or other means, to map physical data model with logical data model which will be used to publish data. Logical data model is aligned with a [conceptual data model](#).
3. DSA tables marked as ready for publishing are converted into JSON Schemas and merged with UAPI OpenAPI specification template to produce final OpenAPI specification, which is published in **API Gateway**.

At the same time list of available authorization scopes are generated from code names specified in DSA table and published to **Authentication and authorization service**.

Data flow

Next **data flow** represents, how actual data moves through all the components:

1. **Data Agent** reads data directly from **Data source** or through a **Data Service**. Data Agent supports multiple data exchange protocols and formats.

Data Agent uses DSA table to transform data between physical and logical data models and between source protocol and format into standardized UAPI data exchange protocol using JSON format.

2. **Data Agent** performs data authorization, using authorization scopes. Authorization scopes are validated with **Auth Service**. Using authorization scopes and DSA, Data Agent applies data filters in order to return only those data objects, which are available for a specific data user.

3. **API Gateway** works as a reverse proxy and load balancer, accepts requests from Clients, validates these requests against OpenAPI schemas received from **Data Catalog**, validates authorization tokens with **Auth service**, provides request logging, rate limiting and other services.

API Gateway after initial validation, passed requests to **Data Agent**, which reads data from primary sources and provides protocol and format conversions.

4. **Client** authenticates using **Auth service** and receives access token, which authorization scopes. Using access token, Client makes request to API Gateway by providing access token and receives data or error response if something goes wrong.

API URL structure

All API URL path elements are as defined in [xsd:token](#). They are constructed using codenames defined in [DSA](#) and follow these patterns:

- Get list of objects:

```
https://  
domain /  
group /  
form /  
publisher /  
is /  
service /  
version /  
dataset /  
model
```

- Get single object:

https://
domain /
group /
form /
publisher /
is /
service /
version /
dataset /
model /
id

- Get an older version of a single object:

https://
domain /
group /
form /
publisher /
is /
service /
version /
dataset /
model /
id /
rev/
revision

- Get single object property value:

https://
domain /
group /
form /
publisher /
is /
service /
version /
dataset /
model /
property

https://
domain /
group /
form /
publisher /

is /
service /
version /
dataset /
model /
id /
property

- Perform a custom action on a collection or on a single object:

https://
domain /
group /
form /
publisher /
is /
service /
version /
dataset /
model /
action

https://
domain /
group /
form /
publisher /
is /
service /
version /
dataset /
model /
id /
action

- Standard query parameters *MUST* be used:

https://
domain /
group /
form /
publisher /
is /
service /
version /
dataset /

model ?

query

https://

domain /

group /

form /

publisher /

is /

service /

version /

dataset /

model /

id ?

query

- Retrieve object or property based on scope:

https://

domain /

group /

form /

publisher /

is /

service /

version /

dataset /

model /

scope

https://

domain /

group /

form /

publisher /

is /

service /

version /

dataset /

model /

id /

scope

- The following pattern *MAY* be used for APIs, mostly legacy APIs, that do not support the standard UAPI exchange protocol:

https://

domain /

services/
form /
publisher /
is /
service /
version /

This pattern is a valid alternative in the current version of the specification, but it *SHOULD* be considered deprecated in future versions.

- Additional API specific endpoints
 - Get the version of the API and backend infrastructure:

https://
domain /
version

- Get health status of the service and underlying infrastructure"

https://
domain /
health

- Ownership and naming

State Digital Solution Agency has the exclusive right to register **publisher** and **is** values in the data.gov.lt catalog and to resolve disputes among Public Organizations.

An Agent has the right to choose **service**, **dataset**, **model**, and **property** values, provided that these values conform to the syntax rules of this and [DSA specification](#).

Meaning of dynamic URL parts:

- domain - domain of the data provision platform. Depending on selected data access mode:
 - **open** - Lithuanian Open Data Vault ([get.data.gov.lt](#))
 - **public** - External Gateway ([api.gov.lt](#))
 - **protected** - Internal Gateway ([apigw.gov.lt](#))
 - **private** - direct IS data access domain

- group - Logical grouping of data services. Available values:

datasets — UAPI data services based on Data Structures published in Data Catalog.

services — mostly legacy APIs, that do not support the standard UAPI exchange protocol

- form - The legal form of an organization. This is a controlled vocabulary with only two permitted values:

gov — government sector

com — private or other non-government sector registered Legal Entities

The **gov** value is understood as being under the regulation of the Law on the Management of State Information Resources of the Republic of Lithuania.

The **com** value covers all other cases.

- publisher - A short code name of an organization or agent. It represents the class **foaf:Agent** in cases when **form=gov** the definition is more narrow and would represent **cv:PublicOrganisation** (**dc:publisher**) and its property **skos:notation**.
- is - IS is a short code identifier representing an information system, that provides data. In the context of DCAT-AP-LT v2 most often representing the **dcatapl:InformationSystem** class and its property **skos:notation**, can also represent sub information system of an information system.
- is value used by existing data information system:

isris — Lithuanian Protected Data Catalog

Example:

```
.../datasets/gov/rc/jadis/at280/1/at280_israsas/DalyvioAsmensIsrasas
```

In this example:

datasets - identifies it as part of datasets group,

gov - identifies it as government sector data,

rc - is the code name of publisher organisation,

jadis - is understood as an information system,

at280 - is the data service name,

1 - is the version of the service,

at280_israsas - is the name of the dataset,

DalyvioAsmensIsrasas - is the name of the model.

- service - A short code name of a data service. It represents the class **dcat:DataService** and its property **skos:notation**.
- version - The data schema version of a data service. It represents the MAJOR version number according to the **Semantic versioning v2.0.0** version management rules.

The following rules apply:

a. If a [dcat:Dataset](#) receives a new MAJOR [version](#), it does not affect the [dcat:DataService](#) URI; it only affects the [dcat:downloadURL](#) of the [dcat:DataService](#).

b. If a [dcat:Dataset](#) does not receive a new MAJOR version, but the [dcat:DataService](#) that [dcat:servesDataset](#) that [dcat:Dataset](#) receives

a MAJOR update of the UAPI specification, then the MAJOR version number used in the URI version segment *MUST* be updated accordingly.

- dataset - A short code name of a specific dataset from a catalog. It *MAY* also represent a sub-dataset.

It represents the class [dcat:Dataset](#) and its property [skos:notation](#).

When defining a sub-dataset a [broader_dataset/narrower_dataset](#) structure *SHOULD* be used.

- model - A SHACL validation shape ([shacl:NodeShape](#)) that defines constraints for a set of objects conforming to the same data model, with [shacl:targetClass](#) pointing to the corresponding [rdfs:Class](#).
- id - A unique global object identifier. UUID format is RECOMMENDED.
We provide recommendation for identifiers:

model	property	type	ref	description
Int			id	
	_id	integer		/id
	_revision	integer		/id/version
	id	integer		
	version	integer		
Uuid			uuid	
	_id	uuid		/uuid
	_revision	uuid		/uuid/version
	uuid	string		
	version	string		
Comp			id,code	
	_id	string		/id,code
	_revision	string		/id,code/major,minor
	id	integer		
	code	string		
	major	integer		

model	property	type	ref	description
	minor	integer		
Str			name	
	_id	string		/=name
	_revision	string		/=name/=version
	name	string		
	version	string		
Base32			code	
	_id	base32		/=BASE32(code)
	_revision	string		/=name/=version
	code	string		
CompBase32			id,code	
	_id	base32		/=BASE32(CBOR(id,code))
	id	string		
	code	string		

More detailed explanation provided in [Identifier](#)

- revision - A unique identifier of a historical object version.
Revision semantics *MUST* follow the Linked Data Event Streams ([LDES](#)) model. UUID format is *RECOMMENDED*. Attributes are defined in [DCAT Version](#) and [_revision](#) is defined in [VSSA Core Vocabulary](#)
- property - A subresource used to retrieve the value of a property, especially where the property value is a large object, linked file, or array. It represents an RDF property, such as [owl:ObjectProperty](#) or [owl:DatatypeProperty](#). More details in [Subresource](#)

Example:

```
<owl:ObjectProperty rdf:about="#hasBankAccount"/>
```

Example URI:

```
/datasets/com/cemety/ldis/1/kapines/vilnius/Cemeteries/123/has_label
```

- action - An operation performed on a collection of objects or on a single object. It *MUST* start with `..`.
More detailed explanation provided in [Action](#)
- scope - Access based filter using OAuth 2.0 scopes. It *MUST* start with `@`. It is a custom `scope` that filters data on a access token level. More details in [Scope](#).

Static parts:

- `rev` - revision endpoint to return older versions of the object.

Example API request structure and what it returns:

`/dataset/`

model - get all objects of a Model

`/dataset/`

model?_

reserved=

value - filtered objects by reserved keywords, like `_id`, `_type`, `_select`, ...

`/dataset/`

model?

property=

value - filtered objects by Model properties

`/dataset/`

model/

property - get all dspecified property values of all objects

`/dataset/`

model

`/:changes` - get changes of all Model objects

`/dataset/`

model/

action - custom action on all Model objects

`/dataset/`

Model/

scope - get all Model objects filterred by custom token level filter

`/dataset/`

model/

reference - get one Model object based on it's reference

`/dataset/`

model/

reference/

property - get one property value of a specific object or all properties of file or array types

/dataset/
model/
reference/:changes - get changes of a single object

/dataset/
concept/
reference/
action - custom action on a single object

/dataset/
concept/
reference/rev/
revision - get a specific version of an object

The following URL components are mandatory for all datasets and services endpoints:

- form
- publisher
- is
- service

All other components are optional and depend on the level of access being addressed, such as:

- dataset
- model
- revision
- property
- action

Download URL example:

```
https://get.data.gov.lt/datasets/gov/rc/ar/uapi/text_with_coordinates/Address/e96cc0cc-08be-460d-a887-98f80612a402
```

When interpreting URL, there are few distinctions in each component, that might help separate components presented in URL:

- Namespace components form, publisher, is, service and dataset must start with lowercase ascii letter and must always follow one after the other, with IS exception, where dataset might be used without IS.

- model name must always start with uppercase ascii letter and can be used to separate namespace components from model.
- property must always start with lowercase ascii letter and cannot contain - character to separate property from id.
- revision is a UUID value.
- action must always start with :.
- scope must always start with @.

Semantic interpretation of API URLs

The API URL structure defines an access path and governance namespace. It does not, by itself, define an RDF graph structure.

The semantic meaning of URL segments is expressed through [JSON-LD Context](#) and [RDF](#) mappings. In particular:

[publisher](#) identifies a [foaf:Agent](#)

[is](#) identifies a [dcatapl:InformationSystem](#)

[service](#) identifies a [dcat:DataService](#)

[dataset](#) identifies a [dcat:Dataset](#)

[model](#) identifies an [owl:Class](#)

[property](#) identifies an [rdf:Property](#).

URI and Semantic Resource Identification Overview

This specification distinguishes between:

vocabulary resources, such as [classes](#), [properties](#), and [concepts](#);

and

entity resources, such as identified instances of [datasets](#), [is](#) entries, and [domain](#) objects.

These resource types *MUST* be identified using consistent URI patterns.

- Vocabulary resources

Vocabulary resources define the semantic layer of the system, including ontologies and controlled vocabularies.

Vocabulary resources *SHOULD* use the following pattern:

[https://data.gov.lt/id/
vocab/](https://data.gov.lt/id/vocab/)

termType/
termName

Where:

- vocab — a code name of the data domain; it *MAY* align with EU or W3C vocabulary names and corresponds to prof.hasToken
- termType — type of vocabulary term; *MUST* be one of:
 - class
 - property
 - concept
- termName — local name of the term

Examples

<https://data.gov.lt/id/jar/class/JuridinisAsmuo>

<https://data.gov.lt/id/jar/property/kodas>

<https://data.gov.lt/id/evrk/concept/6201>

- Entity resources

Entity resources represent identifiable instances of real-world or digital objects.

They follow [10 rules for persistent URIs](#)

```
https://{domain}/{type}/{concept}/{reference}
```

In case of UAPI **type=id** is used. The namespace of the URI is defined in a Conceptual Data Structure Definition document (KDSA) provided with Information System metadata in Data Catalog.

Entity resources *MUST* use the following identifier pattern:

<https://data.gov.lt/id/>
vocab/
entityType/
id

Where:

- vocab — a code name of the data domain
- entityType — a semantic identifier of the entity type (typically aligned with an owl:Class)
- id — a globally unique and persistent identifier of the entity

Versioned entity resources

If versioning is required, entity resources *MAY* include a revision identifier:

`https://data.gov.lt/id/{vocab}/{entityType}/{id}/{revision}`

`https://data.gov.lt/id/
vocab/
entityType/
id/
revision`

Where:

- revision — an identifier of a historical version of the entity

Revision semantics *MUST* follow ([DCAT Version Info](#) model).

More info in [Revision](#).

Relationship with API URLs

API URLs define an access path and governance namespace, while URIs define persistent semantic identifiers.

API URL *MUST* resolve to a representation, but URI *MAY* de-reference to such representation.

For example:

```
GET  
/datasets/{form}/{publisher}/{is}/{service}/{version}/{dataset}/{model}/{id}  
HTTP/1.1  
Host: api.gov.lt  
Accept: application/ld+json
```

Response:

```
{  
  "@context": "https://data.gov.lt/context/{dataset}/{version}.jsonld",  
  "@id": "https://data.gov.lt/id/{vocab}/{entityType}/{id}",  
  "_id": "{id}",  
  "_revision": "{revision}"  
}
```

If a specific historical version is requested, the `@id` *MAY* include the revision segment:

```
{  
  "@id": "https://data.gov.lt/id/{vocab}/{entityType}/{id}/{revision}"  
}
```


- URIs are global, persistent identifiers and *MUST NOT* change over time.
- API URLs are an access mechanisms and *MAY* change without affecting the identity of resources.
- The URI structure *MUST NOT* encode transport-level or routing-specific information.

Concepts

Service

Data exchange between systems can be achieved in a lot of different ways. In the context of data integrations and exchange via Intranet or Internet and in the context of UAPI the main focus is on API data exchange.

In this case Service would be defined as "Machine to Machine Interface" Application Presentation as per [EIRA](#).

This type of API presentation *SHOULD* be the primary focus for all Lithuanian government IT system modernization efforts as per [DTFS](#).

To differentiate between specific usecases of data provision, different services *MAY* be defined for a particular IS.

Identifier

Global object identifiers are assigned to real or fictional world entities. For example, there can be a building in real world, global identifier would be assigned to the building as a real object.

That means, there can't be two different global identifiers, pointing to the same building in any dataset.

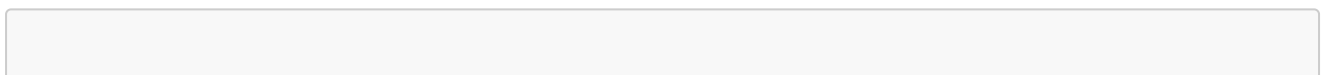
Different datasets and different models might have multiple local identifiers assigned by a database management system or a registry, all local identifiers must be mapped to a single global identifier.

When exchanging data, objects are only identified using global identifiers.

There can be multiple separate datasets and multiple models providing data form same building, but the global identifier must be the same for all datasets.

Global identifiers must be assigned to all objects using a reserved property name `_id`.

For example we can have multiple datasets containing data about the same building, since building is the same, single global identifier must be used:



```
/datasets/gov/rc/ar/uapi/text_with_coordinates/Address/e96cc0cc-08be-460d-a887-98f80612a402  
/datasets/gov/rc/ntr/uapi/report/Pastatas/e96cc0cc-08be-460d-a887-98f80612a402
```

Model

Real world entities are identified using global identifiers. Each real world entity might have different data shape, form or schema.

Single real world entity might be represented using multiple different data models using different set of properties, but different models must share the same global identifier for the same entity.

A model is just a data schema for an entity.

Model name is provided by the reserved `_type` property:

```
{  
  "_type": "/datasets/gov/rc/ar/uapi/text_with_coordinates/Address",  
  "_id": "e96cc0cc-08be-460d-a887-98f80612a402"  
}
```

Object

A single data entity provided by a specific data model is called object.

Objects are associate with data models, when we reference we must provide global identifier and model, to know the entity and data model used to describe the entity.

Object must have at least `_type` and `_id` properties:

```
{  
  "_type": "/datasets/gov/rc/ar/uapi/text_with_coordinates/Address",  
  "_id": "e96cc0cc-08be-460d-a887-98f80612a402"  
}
```

Reference

Reference is an object property pointing to another object. Reference must provide global identifier of a referenced object.

For example:

```
{
  "_type": "/datasets/gov/rc/ar/uapi/text_with_coordinates/Country",
  "_id": "e96cc0cc-08be-460d-a887-98f80612a402",
  "name": "Lithuania",
  "capital": {
    "_id": "60b9bafd-dac7-4579-a435-31c80b593384"
  }
}
```

Here, **capital** is a property of `/datasets/gov/rc/ar/uapi/text_with_coordinates/Country` model and is a reference to another object.

Resource

Resource is a document, containing one or multiple objects.

For example:

```
{
  "_type": "/datasets/gov/rc/ar/uapi/text_with_coordinates/Country",
  "_id": "e96cc0cc-08be-460d-a887-98f80612a402",
  "name": "Lithuania",
  "capital": {
    "_id": "60b9bafd-dac7-4579-a435-31c80b593384",
    "name": "Vilnius"
  }
}
```

Resource above, has two objects.

Subresource

Some types of properties might have subresources in order to separately return large blobs of data.

For example if `/datasets/gov/rc/ar/uapi/text_with_coordinates/Country` model has a **flag** property of **file** type, then file content must be returned separately:

```
GET /datasets/gov/rc/ar/uapi/text_with_coordinates/Country/e96cc0cc-08be-460d-a887-98f80612a402
```

```
{
  "_type": "/datasets/gov/rc/ar/uapi/text_with_coordinates/Country",
  "_id": "e96cc0cc-08be-460d-a887-98f80612a402",
  "name": "Lithuania",
  "flag": {
    "_content_type": "image/png",
    "_name": "flag.png",
    "_size": "1584684"
  }
}
```

When we do a resource call, we don't get file content, in order to get file content we must do a separate API call to a subresource:

```
GET /datasets/gov/rc/ar/uapi/text_with_coordinates/Country/e96cc0cc-08be-460d-a887-98f80612a402/flag
```

And this will return raw image data as `image/png` content type.

Action

An action performed with multiple objects or a single object. Action name in a URI is denoted with a `:` symbol, for example:

```
GET /datasets/gov/rc/ar/uapi/text_with_coordinates/Country/actions/:changes
```

In the example above action name is `changes` and it lists all changes made to `Country` objects. This is an action, that works on multiple objects.

But also, there can be an action on a single object:

```
GET /datasets/gov/rc/ar/uapi/text_with_coordinates/Country/e96cc0cc-08be-460d-a887-98f80612a402/actions/:changes
```

There is a list of build-in reserved implicit actions, where you don't need to specify action name:

- `GET /.../ model (:getall)` - get list of objects.
- `GET /.../ model / id (:getone)` - get a single object.
- `GET /.../ model ? query (:search)` - get list of objects with a filter.
- `POST /.../ model (:create)` - create new object.
- `PUT /.../ model / id (:update)` - rewrite whole existing object.
- `PATCH /.../ model / id (:path)` - update some properties of existing object.

- **DELETE** `/.../ model / id (:delete)` - delete an existing object.

Also there are some built-in reserved explicit actions:

- **GET** `/.../ model /:changes` - get list of changes to all model objects.
- **GET** `/.../ model / id /:changes` - get list of changes to a single object.
- **DELETE** `/.../ model / id /:wipe` - completely remove object without storing changes in changelog.

Data agents might implement custom actions.

Version

Service version is provided after the service name in the URL:

```
/datasets/gov/rc/ar/uapi/2/text_with_coordinates/Country
```

If version component is not given, then latest service version is used.

Version number is a single positive integer number specifying major version of the service.

Revision

All objects must have a revision number provided as reserved `_revision` property name.

Revision is a string, that identifies object version. This might be a UUID string.

When update action is made on an object, revision number must always be provided, in order to compare object version received by the Client and object version in the Resource. This check is needed to avoid conflicts, when two or more concurrent requests try to update same object.

When object is changed, a new revision number must also be generated.

Revision number is also used, references older object version if Resource stores older object versions:

```
/datasets/gov/rc/ar/uapi/text_with_coordinates/Country/e96cc0cc-08be-460d-a887-98f80612a402/dd22f1b4-09c2-48ee-bf7a-7bf082da9940
```

Second UUID number is a revision and following request retrieves an older object version.

Revision number is retrieved as part of the `changes` request for the object.

Revision number is also used as [HTTP ETag](#)

[header](#) to

indicate if resource is outdated.

Maturity

Metadata and data maturity is defined in [DSA](#).

It is represented as `x-level` property of a model or property.

```
{
  "components": {
    "schemas": {
      "City": {
        "type": "object",
        "x-level": 4,
        "properties": {
          "country": {
            "type": "object",
            "x-level": 3
          }
        }
      }
    }
  }
}
```

To comply to UDS specification maturity level *MUST* be at least 2.

Highest maturity level be *SHOULD* be strived for.

Identifier values might have different forms depending on data maturity level. There are five maturity levels:

- **0** - data or identifier is not available, even if property is present, in such cases all values might be set to `null`. All actions that required identifier are not available for the object.
- **1** - data is not machine readable, usually this means, that there is no clear structure or consistency in data values. For example a number might have different units, but unit itself is not known. Or date and time values might be given using different formats. For identifiers that would mean that there is no clear way to create technical to this object, or object identifier is not a unique value in a model.
- **2** - data is consistent and machine readable, but values are given in a non-standard way. For example integer values, might be given as strings, or date values might be given in a non-standard format. For identifier that would mean that identifier could be a non static value for the object, there is

no link to a base source of the object or reference identifier of a base object is not defined.

- **3** - objects do not have a global object identifier, this usually applies to categorical data, references and `_id` property of an object. In such cases it is not possible to retrieve a single object by its global identifier, `_id` might not be given.
- **4** - objects have global identifiers, but object properties are not linked with a vocabulary terms describing semantic meaning of data.
- **5** - all of the lower level requirements are met.

Data with lower maturity level might require more effort to process, but still data in any form is better, than no data at all. The only thing, that is required, to specify correct maturity level value if data does not meet the requirements.

Maturity level is set and can be found in the Catalog.

According to object maturity API responses MUST return following elements:

== Global ID is available and actions that require identifier performed providing it in the path ==

- **Maturity level 5**

```
HTTP/1.1 200 OK
Content-Type: application/ld+json

{
  "@context":
  "https://data.gov.lt/context/{dataset.id}/{version}.jsonld",
  "@id": "https://data.gov.lt/id/{vocab}/{term}/42",
  "_id": "42",
  "id": 42,
  "title": "Duomuo"
}
```

- **Maturity level 4**

```
HTTP/1.1 200 OK
Content-Type: application/ld+json

{
  "_id": "42",
  "id": 42,
  "title": "Duomuo"
}
```

== All actions that require identifier only available whe it is provided in query ==

- **Maturity level 3**

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": 42,
  "title": "Duomuo"
}
```

- **Maturity level 2**

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "code": 1000,
  "title": "Duomuo"
}
```

- **Maturity level 1**

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "title": "Duomuo"
}
```

== No actions that require identifier are available ==

- **Maturity level 0**

```
HTTP/1.1 200 OK
Content-Type: application/json

[
  {
    "title": "Duomuo"
  }
]
```


Authorization

Authorization must be implemented using OAuth 2.0 protocol.

In order to understand authorization process, first we need to understand all the parties participating in data exchange process:

- **User** - a person (a human being), that has access to data, a person can represent himself or act as an organization representative. In OAuth terms, this is a Resource Owner.
- **Client** - a software (mobile, web or other application), that can act on behalf of a user or itself.
- **Auth** - authentication and authorization server responsible for authenticating users and clients, issues access tokens. In OAuth terms this is a Authorization Server.
- **Catalog** - a data catalog, a place where all information about agents, resource, clients and smart contracts are registered.
- **Resource** - a data service, a database or a data file, place where data are stored.
- **Agent** - a software responsible for data exchange, accepts data queries, reads data from a Resource and returns data via API using UAPI exchange protocol.
- **Gateway** - a reverse proxy, responsible for routing client requests to multiple Agents and performs basic request data and access token validation.

Following authorization flows (grant types) are supported:

- Authorization code - for granting access to a specific User on behalf of a Client.
- Client credentials - for granting access directly to a Client, who acts on its own.

Registration

In order to get access to data, before any data exchange happens, following steps must be completed:

1. An Agent must be installed and configured in order to be able to access Resource data. Agents should be installed on the same infrastructure as Resource.
2. Agent inspects Resource data structure (DSA) and published collected information about data structure to Catalog.

3. Data steward organized data structures (DSA) into datasets, and specifies what data can be use for exchange outside if Resource and published this information for Client developers.
4. Client developers discovers published datasets, registers client and requests access to data they need, specifying list of scopes that are required for the Client. After registration, clients receive client id and client secret assigned by the Authorization server.
5. Resource maintainers reviews data access requests and by approving access creates a smart contract. Information about smart contract is published to Authorization server. Authorization server knows what clients and what data (scopes) can access.
6. Agent and Gateway registers themselves with the Authorization servers and receives public keys from Authorization server needed for access token validation.

When all these steps are completed all parties are ready for data exchange.

Client authorization

For client authorization client credentials flow (or grant type) is used, when client is granted access to data as a system service acting on its own without representing any specific user.

In order to make requests, following steps are needed:

1. Client, uses client id and secret received at client registration time, to get access token from Authorization server:

```
POST /token HTTP/1.1
Host: auth.gov.lt
Authorization: Basic
YjRjNTMyNDktMTQxZC00OTI2LWE5YzQtNjBjNGJiNjVlZWJmOnNlY3JldAo=

grant_type=client_credentials&
resource=https://api.gov.lt/&
scope=uapi:/datasets/gov/rc/ar/uapi/text_with_coordinates/Country/:getall
```

Authorization server returns JWT access token in a following JSON response:

```
{
  "token_type": "Bearer",
  "expires_in": 3600,
  "access_token":
```

```
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJqdGkiOiI3ZWFKODU5MS01ZDUxLTQyNjQtYjFiMi1iNzk0M2UxNDIxNzUiLCJpc3MiOiJodHRwczovL2F1dGguZ292Lmx0LyIsImF1ZCI6Imh0dHBzOi8vYXBpLmdvdi5sdC8iLCJleHAiOjE2Mzk1Mjg5MTIsIm1hdCI6MTYxODM1NDA5MCwic3ViIjoINTMzMmM5Y2U0YmM5O00YzRlLWExMjItYjgxZTVmOTQ5OGZjIiwiaWF0IjE5YXBpOi9kYXRhc2V0cy9nb3YvcmlvYXNvd3MvQ291bnRyeS86Z2V0YWxsIn0.CcvXCq7_iSpWQJR5N_cGficA6EmDtQZoGxdaToA70G0",
  "scope":
  "uapi:/datasets/gov/rc/ar/uapi/text_with_coordinates/Country/:getall"
}
```

Payload of JWT token (`access_token` value) must conform to [RFC 9068: JWT](#)

[Profile for OAuth 2.0 Access](#)

[Tokens](#) and should look like

this:

```
{
  "jti": "7ead8591-5d51-4264-b1b2-b7943e142175",
  "iss": "https://auth.gov.lt/",
  "aud": "https://api.gov.lt/",
  "exp": 1639528912,
  "iat": 1618354090,
  "sub": "b4c53249-141d-4926-a9c4-60c4bb65eebf",
  "client_id": "b4c53249-141d-4926-a9c4-60c4bb65eebf",
  "scope":
  "uapi:/datasets/gov/rc/ar/uapi/text_with_coordinates/Country/:getall"
}
```

When client acts on behalf of itself, `sub` and `client_id` must be the same.

2. Client then uses received access token, to get data like this:

```
GET
https://get.data.gov.lt/datasets/gov/rc/ar/uapi/text_with_coordinates/Cou
ntry HTTP/1.1
Host: api.gov.lt
Authorization: Bearer
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJqdGkiOiI3ZWFKODU5MS01ZDUxLTQyNjQtYjFiMi1iNzk0M2UxNDIxNzUiLCJpc3MiOiJodHRwczovL2F1dGguZ292Lmx0LyIsImF1ZCI6Imh0dHBzOi8vYXBpLmdvdi5sdC8iLCJleHAiOjE2Mzk1Mjg5MTIsIm1hdCI6MTYxODM1NDA5MCwic3ViIjoINTMzMmM5Y2U0YmM5O00YzRlLWExMjItYjgxZTVmOTQ5OGZjIiwiaWF0IjE5YXBpOi9kYXRhc2V0cy9nb3YvcmlvYXNvd3MvQ291bnRyeS86Z2V0YWxsIn0.CcvXCq7_iSpWQJR5N_cGficA6EmDtQZoGxdaToA70G0
```

Request must be made to API Gateway, which validate access token using public key received from Authorization server and validates request data

using OpenAPI schema received from Catalog.

3. Gateway passes Client request to Agent, which again repeats token and request data validation.

In addition to access token validation performed on Gateway, Agent interprets given scope values and ensures, that Client has access to the requested Resource.

Agent might require User authorization level token and refuse request if client acts on itself.

4. Finally Agent transforms an UAPI request into a protocol, that is understood by a Resource, retrieves data and transforms result back to UAPI.

User authorization

For user authorization authorization code flow (or grant type) is used.

1. A User (a human being) uses a Client application in order to access a Resource.
2. Client redirects User to Authorization server, where User can authenticate and allow Client to act on behalf of User.

When redirecting, following URL is constructed to the Authorization server:

```
https://auth.gov.lt/auth?  
  response_type=code&  
  client_id=b4c53249-141d-4926-a9c4-60c4bb65eebf&  
  redirect_uri=https://app.example.com/auth&  
  
  scope=uapi:/datasets/gov/rc/ar/uapi/text_with_coordinates/Country/:getall  
&  
  state=xcoiv98y2kd22vusuye3kch
```

3. Authorization server asks User to identify himself by providing user name and password or other means of authentication.

After authentication, User is asked if he agrees to authorize Client application with scopes request by the Client application.

If agreed, then Authorization server redirects User back to the Client application with following URL:

```
https://app.example.com/auth?  
  response_type=code&
```

```
state=xcoiv98y2kd22vusuye3kch&
code=g0ZGZmNjVmOWIjNTk2NTk4ZTYyZGI3
```

- After successful User authentication, Client receives authorization code (`code`) from Authorization server.

Client uses authorization code to get access token by making following request to Authorization server:

```
POST /token HTTP/1.1
Host: auth.gov.lt
Authorization: Basic
YjRjNTMyNDktMTQxZC00OTI2LWE5YzQtNjBjNGJiNjVlZWJmOnNlY3JldAo=

grant_type=authorization_code&
code=g0ZGZmNjVmOWIjNTk2NTk4ZTYyZGI3&
resource=https://api.gov.lt/&
scope=uapi:/datasets/gov/rc/ar/uapi/text_with_coordinates/Country/:getall
```

Authorization server returns JWT access token in a following JSON response:

```
{
  "token_type": "Bearer",
  "expires_in": 3600,
  "access_token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJqdGkiOiI3ZWFKODU5MS01ZDUxLTQyNjQyYjFiMi1iNzk0M2UxNDIxNzUiLCJpc3MiOiJodHRwczovL2F1dGguZ292Lmx0LyIsImF1ZCI6Imh0dHBzOi8vYXBpLmdvdi5sdC8iLCJleHAiOjE2Mzk1Mjg5MTIsImh0dCI6MTYxODM1NDAMCwic3ViOiJoInDA2M2YxYzAtN2Q4NC00MzBjLTliZDEtMWE0YmM5ZjdhdWZlIiwiaWY2xpZW50X2lkIjoiaYjRjNTMyNDktMTQxZC00OTI2LWE5YzQtNjBjNGJiNjVlZWJmIiwic2NvcGU0iOiJ1YXBpOi9kYXRhc2V0cy9nb3Yvcmlvd3MvQ291bnRyeS86Z2V0YXsIn0.I3BQ41W_-BHtrxBOPF_wmE92gfxI9T7DPCqcSZe7-v8",
  "scope":
"uapi:/datasets/gov/rc/ar/uapi/text_with_coordinates/Country/:getall"
}
```

Payload of JWT token (`access_token` value) must conform to [RFC 9068: JWT](#)

Profile for OAuth 2.0 Access

Tokens and should look like

this:

```
{
  "jti": "7ead8591-5d51-4264-b1b2-b7943e142175",
  "iss": "https://auth.gov.lt/",
  "aud": "https://api.gov.lt/",
  "exp": 1639528912,
```

```
"iat": 1618354090,  
"sub": "4063f1c0-7d84-430c-9bd1-1a4bc9f7aeae",  
"client_id": "b4c53249-141d-4926-a9c4-60c4bb65eebf",  
"scope":  
  "uapi:/datasets/gov/rc/ar/uapi/text_with_coordinates/Country/:getall"  
}
```

When client acts on behalf of a User, **sub** is a global identifier of a representative, which is also must be used by the Resource in data, to reference a representative.

sub references not the User itself, but a representative. Representative can represent an organization or a User itself. This distinction is needed, because Resource server might give different permissions of an employee of an organization and for a person not associated with an organization.

Row level access scope might use representative identifier (**sub**) in order to filter data that is only available for the representative.

Authorization server does not know which scopes are available for a specific User, Authorization server only knows which scopes are available for the Client. Even if a token is issued with administrator permissions, Agent does additional validation, by checking if representative **sub** has permission to use this scope.

Scope alone does not grant permission, because Agent validates if representative is allowed to use this scope.

5. All other steps are the same as described for the [Client authorization](#).

Scope

OAuth scope is a string granting permission to perform an action on Resource data.

Scopes are generated in a similar way as API paths using following form:

```
uapi:/datasets/  
form /  
publisher /  
is /  
service /  
dataset /  
model /  
property @  
scope :  
action
```

Meaning of all the components in the scope is the same as described in [URL schema](#), with some additions and differences described below:

- `property` - `property` is used to grant access only to specific properties of a model.
- `@scope` - this is a row-level filter, filter is defined in Catalog and executed by the Agent. Filters optionally might use Representative `sub` identifier received from access token, to make dynamic filtering for a specific representative.

Scope is interpreted by the Agent in a dynamic way.

Scope can take a broad form, for example:

```
uapi::getall
```

Granting access to `getall` action for all datasets and all models.

But also scope can take a narrow form:

```
uapi:/datasets/gov/rc/ar/uapi/text_with_coordinates/Country/:getall
```

Scope can grant access to all data or with a row-level filter, for example:

```
uapi:/datasets/gov/rc/ar/uapi/text_with_coordinates/Country/@resident/:getall
```

`@resident` is a row-level filter, that grants access only to residents of that country.

Row-level filters might require User authorization flow in order to get representative `sub` identifier, which might be used when filtering data.

Query

Actions might use URL query to filter, limit, sort or otherwise influence result returned by an action.

URL consists of following components (there are more components, but for simplicity reasons in this example are only listed main ones):

scheme `://`

host `/`

path ?

query

Query has name and value pairs called parameters:

name =

value

There can be multiple query parameters separated by & symbol.

param &

param

Parameter names can have one of two forms:

- Reserved names - all names starting with `_` are reserved and has a function or a predefined meaning.
- Property names - all names not starting with `_` are interpreted as model property names and can be used to filter data by a property value.

Parameter names can have attributes:

name .

attribute =

value &

There can be none, one or more than one attribute.

For query examples following data model will be used.

```
classDiagram
    direction LR
    class Country {
        code: string
        name: string
        population: integer
    }
    class City {
        name: string
    }
    Country <-- City : country
```

Filter

Simple filtering can be done like this:

```
Country?code=lt
```


This will filter countries by `code`.

It is also possible to filter using different comparison operators, for example:

```
Country?population._gt=1000
```

List of all available operators:

- `_gt` - greater than
- `_ge` - greater than or equal
- `_lt` - less than
- `_le` - less than or equal
- `_sw` - starts with
- `_ew` - ends with
- `_co` - contains

Multiple parameters can be added into query:

```
Country?code=lt&population._gt=1000
```

Logical connectives

By default all filtering expressions are interpreted as logical conjunctions (AND), as indicated by query parameter separator `&` character.

`_or` connects previous parameter into a logical disjunction (OR), for example:

```
Country?code=lt&_or.code=lv
```

Will produce `(code=lt OR code=lv)` query expression.

`_and` connects previous parameter into a logical conjunction (AND):

```
Country?code=lt&_and.name=ltu&_or.code=lv&_and.name=lat
```

This will produce `(code=lt AND name=ltu) OR (code=lv AND name=lat)`.

`&` operator always takes precedence, and if `_and` would not be added:

```
Country?code=lt&name=ltu&_or.code=lv&name=lat
```

We would get `code=lt AND (name=ltu OR code=lv) AND name=lat` expression, which would be a completely different thing.

If `_and` or `_or` is used, it adds grouping to filter expressions, for example:

```
Country?code=lt&_and.name=ltu&code=lv&_and.name=lat
```

Will produce `(code=lt AND name=ltu) AND (code=lv AND name=lat)`.

`_and` and `_or` can be used multiple times in sequence, to add multiple parameters into a logical connective group:

```
Country?code=lt&_or.code=lv&_or.code=ee
```

Will produce `(code=lt OR code=lv OR code=ee)` expression.

Select

By default, all model properties are returned, but it is possible to select only specific properties using `_select`:

```
Country?_select=code,name
```

This will return data containing only `code` and `name` properties.

Join

Some Agents might provide joining capabilities, allowing to retrieve data from multiple models with a single request.

Usually joins are allowed only if data physically stored on the same data management system, but more advanced Agents might allow to join models from different datasets stored in separately.

A join can be requested by specifying attribute of a referenced object, for example:

```
City?_select=country.name,name
```

This will select data from both `City` and `Country` models, because `City/country` property is a Reference to the `Country` model.

Joins can be used with for selecting, filtering, sorting and other places, where a property can be specified.

Sort

By default objects are sorted by `_id`, but you can change order with `_sort`:

```
Country?_sort=name
```

By default sorting is done in ascending (a-z) order, but this can also be changed:

```
Country?_sort=-name
```

You can sort by multiple properties in different directions:

```
Country?_sort=-name,+code,population
```

Limit

By default all objects are returned, but you can limit number of objects returned with `_limit`:

```
Country?_limit=10
```

This will limit result set to 10 objects.

Count

You can get total number of available objects with `_count`:

```
Country?_count=
```

Page

If number of objects is too large to get in a single request, you can use `_page` in combination with `_limit` and `_sort` to get all data in multiple pages.

By default pagination is enabled if you specify `_limit`:

```
Country?_limit=10
```

This will enable pagination and query above is equivalent to:

```
Country?_sort=_id&_limit=10
```

Objects are ordered by `_id` by default, so we don't need to specify `_sort`, unless we want to change pagination key:

```
Country?_sort=code&_limit=10
```

When `_limit` is given, response document will include `_next` attribute pointing to the next page:

```
{
  "_data": [...],
  "_next": "ImE3N2IyNWU1LWUzMtMDMNGE4ZS04Y2YzLTl1ZmMyNWVlMjI0NiIK"
}
```

You can get next page with `_page` query parameter:

```
Country?
_sort=code&_limit=10&_page=ImE3N2IyNWU1LWUzMtMDMNGE4ZS04Y2YzLTl1ZmMyNWVlMjI0NiIK
```

`_page` value is a base64 encoded (with `+/=` characters replaced with `-_.` to make it user safe) JSON string. In this case, decoded `_page` value is:

```
"a77b25e5-e303-4a8e-8cf3-9efc25ee2246"
```

If result is sorted by more than one property, then `_page` JSON value will be an array, for example if we have:

```
Country?_sort=_id,code&_limit=10
```

Then decoded `_page` value will be:

```
["a77b25e5-e303-4a8e-8cf3-9efc25ee2246", "lt"]
```

Pagination might be enabled by default, by the Agent, this will be indicated in the result data:

```
{
  "_data": [...],
  "_next": "ImE3N2IyNWU1LWUzMMDMtNGE4ZS04Y2YzLTl1ZmMyNWVlMjI0NiIK",
  "_limit": 1000
}
```

Presence of `_limit` and `_next` in the result, indicates, that limit is enforced by the Agent.

Tracking data changes

When an object data is created or changed a log is being created and updated to keep track of all of these changes.

This can be retrieved via `change` actions. It can also be used to determine the progression of object changes in time.

This log only store the change that has been done with the object and a revision id `rev` that can be used to retrieve

entire object at that point in time. Change id `cid` is used to determine the change order for the objects, it starts at `0` when the object is created and is increased incrementally with each change that is done.

Important to note that the log should not be used to retain or retrieve historic object data since it only stores the changes.

Object revision retrieval should be used instead.

To retrieve the latest change of the object `-1` can be used as the `cid`:

```
/datasets/gov/rc/ar/uapi/text_with_coordinates/Country/actions/:changes/-1
```

Changes actions are implemented at the model and object levels and conforms to [LDES](#)

For this it uses the [JSON-LD Context](#)

Depending on the size response might be paginated and accessed via query `?_page=n` where `n` is the page (node) number starting with `1`

Data types

ref

file

Specification changes

Draft of 2.0 (2026-05-19)

In [draft](#) 2026-05-19:

Changed URL schema with addition of service and service version

1.1 (2024-11-26)

In [1.1](#) 2024-11-26:

Added LDES support and context

1.0 (2024-10-22)

In [1.0](#) 2024-10-20:

First release of UAPI specification